

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ VITI
Nom d'usage ▶ /
Prénom ▶ THÉO
Adresse ▶ 30 BOULEVARD CAMILLE PELLETAN
13500 MARTIGUES

Titre professionnel visé

Développeur Web et Web Mobile (DWWM)

Code RNCP : 37674

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▣ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▣ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▣ une déclaration sur l'honneur à compléter et à signer ;
- ▣ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▣ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- ▶ Installer et configurer son environnement de travail en fonction du projet web ou web mobile p. 5
- ▶ Maquetter des interfaces utilisateur web ou web mobile p. 10
- ▶ Réaliser des interfaces utilisateur statiques web ou web mobile p. 14
- ▶ Développer la partie dynamique des interfaces utilisateur web ou web mobile p. 19

Développer la partie back-end d'une application web ou web mobile sécurisée p. 22

- ▶ Mettre en place une base de données relationnelle p. 22
- ▶ Développer des composants d'accès aux données SQL et NoSQL p. 24
- ▶ Développer des composants métier coté serveur p. 29
- ▶ Documenter le déploiement d'une application dynamique web ou web mobile p. 31

Titres, diplômes, CQP, attestations de formation (facultatif) p. 35

Déclaration sur l'honneur p. 36

Documents illustrant la pratique professionnelle (facultatif) p. 37

Annexes (Si le RC le prévoit) p. 38-40

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

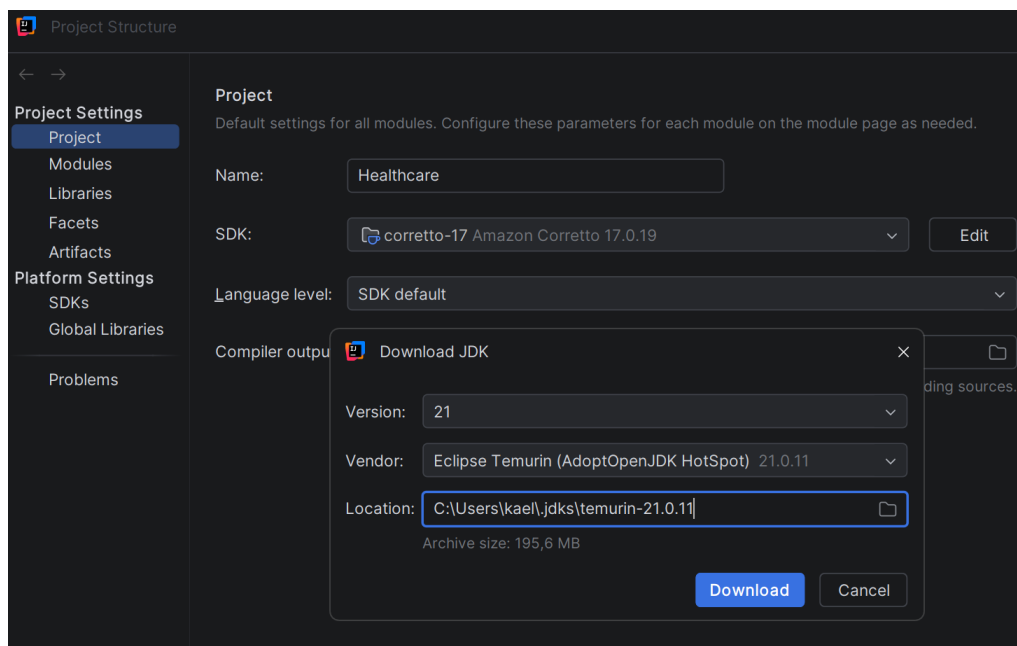
Exemple n°1 ▶ Installer et configurer son environnement de travail en fonction du projet web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de mes projets web, j'ai installé et configuré mon environnement de travail en fonction des besoins techniques des applications que je développe afin de travailler dans les mêmes conditions que mes collègues.

J'ai installé IntelliJ IDEA pour les projets Java et Visual Studio Code pour les autres langages. En parallèle, j'ai effectué une veille sur les environnements de développement disponibles afin de choisir les outils les plus adaptés. J'ai comparé plusieurs IDE, tels que Visual Studio Code, IntelliJ IDEA, Eclipse et NetBeans, puis j'ai retenu IntelliJ IDEA pour les projets Java, notamment grâce à ses fonctionnalités de débogage avancées, son intégration avec Spring Boot et Maven, ainsi que sa compatibilité avec des serveurs d'applications tels que Tomcat. Lorsque je n'utilise pas IntelliJ et que je travaille avec du PHP ou d'autres langages nécessitant un serveur web, j'utilise WAMP afin de disposer d'un environnement local complet incluant Apache et MySQL.

J'ai ensuite installé le JDK (Java Development Kit) afin de compiler et d'exécuter mes applications Java, puis j'ai configuré les variables d'environnement nécessaires au bon fonctionnement du langage sur ma machine.



Project structure (IDE)

J'ai également installé Maven afin de gérer les dépendances, automatiser la compilation et générer les artefacts de déploiement. Une fois l'environnement en place, j'ai personnalisé mon

IDE pour améliorer mon confort de travail, notamment avec l'activation des commentaires TODO, l'ajout d'extensions améliorant la lisibilité, l'ajustement de l'interface et l'utilisation de vues.

Je laisse volontairement l'anglais comme langue par défaut, afin de garantir une meilleure compatibilité avec les outils, frameworks et documentations techniques, ainsi que de faciliter la collaboration avec des interlocuteurs utilisant cette langue internationale.

Pour la création de mes microservices Spring Boot, j'ai utilisé Spring Initializr afin de générer rapidement des projets structurés avec les dépendances nécessaires. Cela m'a permis de partir sur une base propre et cohérente pour chaque service.

J'ai initialisé mes projets avec Git en utilisant la commande `\git init`, puis j'ai mis en place le versionnage du code à travers des commits réguliers pour suivre l'évolution du projet.

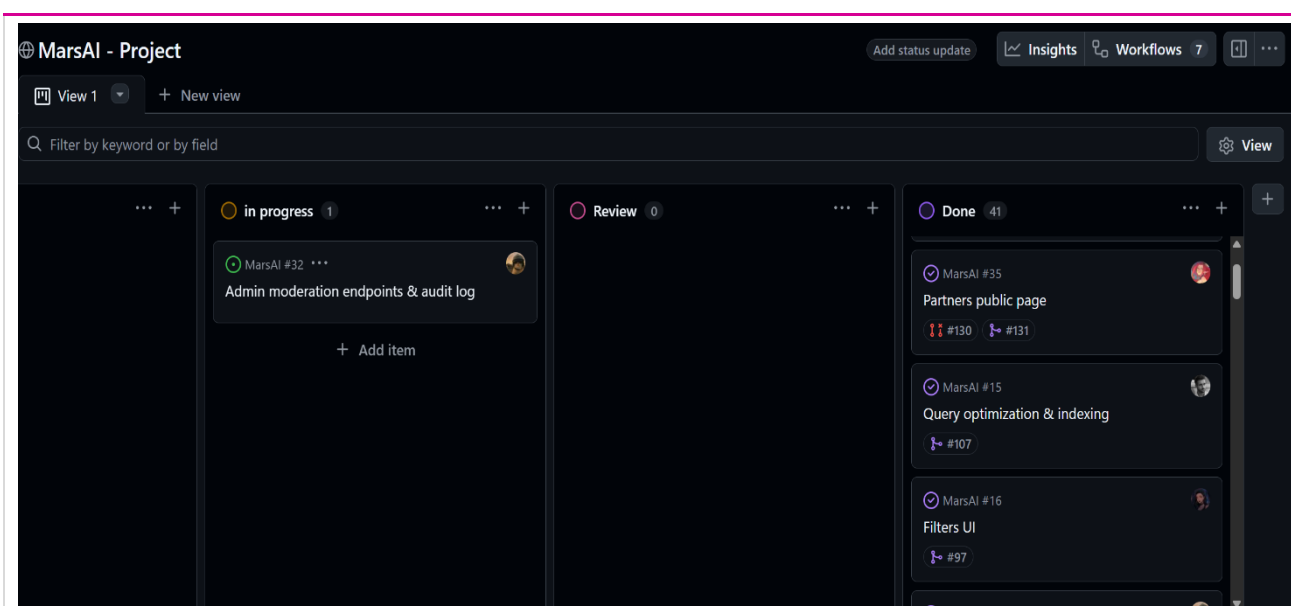
J'ai travaillé avec des branches de type « feature » afin de développer certaines fonctionnalités de manière isolée avant de les fusionner dans la branche principale. Les fusions ont été réalisées à l'aide de commandes Git ou par l'intermédiaire de pull requests, selon les cas.

```
PS C:\Users\kaeve\Documents\MyProjects\fitnetwork\user-service> git add .\src\main\java\com\fitnetwork\user_service\model\User.java
PS C:\Users\kaeve\Documents\MyProjects\fitnetwork\user-service> git commit -m "feat(user): add age validation"
PS C:\Users\kaeve\Documents\MyProjects\fitnetwork\user-service> git push
```

Exemple du workflow : ajout de fichiers, commit, push sur GitHub

J'ai utilisé GitHub pour héberger mes projets et faciliter la collaboration et le suivi du code. J'ai également comparé plusieurs solutions de gestion de version comme GitLab et GitHub, et j'ai retenu GitHub pour sa simplicité et son intégration avec les workflows de développement.

En complément, j'ai utilisé GitHub Projects pour organiser mes tâches sous forme de tableau Kanban afin de suivre l'avancement de mes fonctionnalités.



GitHub Project pour le projet « MarsAi »

Dans le cadre du travail collaboratif, je respecte les conventions de nommage des branches, les stratégies de fusion et les règles définies par l'équipe afin d'intégrer mon travail sans affecter celui des autres développeurs.

Afin de reconstituer sur mon poste de travail un environnement conforme à celui de production, j'utilise Docker (plateforme de conteneurisation) pour créer et exécuter des conteneurs. Je commence par définir une image Docker (modèle contenant l'application, ses dépendances et sa configuration) à l'aide d'un fichier *Dockerfile*. Cette image est ensuite construite (*build*) puis utilisée pour créer un ou plusieurs conteneurs (instances exécutables d'une image).

Cette approche me permet de déployer localement les mêmes types de services que ceux présents en production, tels qu'une base de données ou un microservice, avec une configuration similaire. Les conteneurs étant isolés les uns des autres, ils facilitent les tests, limitent les conflits de dépendances et garantissent que l'application fonctionnera de manière cohérente quel que soit l'environnement d'exécution. Grâce à cette méthode, je peux développer, tester et déployer les services requis dans des conditions proches de la production.

```
PS C:\Users\kaeve\Documents\MyProjects> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
config-server:latest	610442a8ecec	786MB	253MB	
doctors:latest	30751ae79824	830MB	274MB	
eclipse-temurin:17-jre	0d79988c6879	430MB	114MB	
hashicorp/consul:latest	a230dcea0bb1	299MB	83.4MB	

Liste non exhaustive d'images Docker

Afin de reproduire l'environnement de production, j'utilise des images Docker de MySQL (système de gestion de base de données relationnelle) ou de MongoDB (système de gestion de

base de données NoSQL) que je récupère depuis Docker Hub, le registre public d'images Docker. Après le téléchargement d'une image, je crée un conteneur en configurant les paramètres nécessaires dans mon docker-compose.yml tels que notamment les ports, les volumes de persistance et les variables d'environnement. Cette approche facilite le déploiement local des services de données tout en garantissant la cohérence de l'environnement entre les différents postes de développement.

```
mysql:
  image: mysql:8 #Uses the official MySQL version 8 Docker image
  mem_limit: 700m #Limits container memory usage to 700 MB
  container_name: mysql-healthcare_bd #Sets a fixed container name for easier identification
  restart: always
  environment: #Defines environment variables passed to the container
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
  ports:
    - "3308:3306" #Maps host port 3308 → container port 3306 (default MySQL port)
  volumes: #Persists database data outside the container lifecycle
    - mysql_data:/var/lib/mysql
  networks: #Connects container to a custom Docker network
    - healthcare-network #Allows communication with other services on the same network
  healthcheck:
    test: [ "CMD", "mysqladmin", "ping", "-h", "localhost" ]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 10s
```

Exemple d'un docker-compose.yml

Pour la persistance ou l'échange de données entre applications, j'utilise différents formats de fichiers tels que JSON (JavaScript Object Notation), CSV (Comma-Separated Values), ainsi que des scripts SQL permettant l'importation et l'exportation de données.

Lors de l'installation et de la configuration de mon environnement de travail, je consulte la documentation officielle des outils utilisés. La majorité de ces documentations étant rédigée en anglais, je m'appuie sur elles pour comprendre les procédures d'installation, les options de configuration et les bonnes pratiques recommandées par les éditeurs. Par exemple, j'ai consulté la documentation officielle de MongoDB pour apprendre à utiliser les opérations CRUD.

Afin de maintenir un environnement de travail à jour et sécurisé, j'effectue une veille technologique sur les évolutions des outils utilisés et sur les vulnérabilités pouvant les affecter. Je consulte notamment les notes de version, les documentations officielles et les publications spécialisées afin d'identifier les nouvelles fonctionnalités, les correctifs de sécurité et les bonnes

pratiques à appliquer lors de l'installation et de la configuration de mon environnement de développement.

Enfin, afin de contrôler la qualité du code front-end, j'utilise le validateur du W3C pour vérifier la validité et la sémantique de mes balises HTML, Lighthouse, disponible dans les outils de développement de Google Chrome, afin d'évaluer l'accessibilité, les bonnes pratiques, les performances et le référencement naturel de mes pages web.

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for uploaded file index.html (checked with vnu 26.6.29)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only Options...

Check by file upload Choisir un fichier Aucun fichier choisi

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Check

Document checking completed. No errors or warnings to show.

Used the HTML parser.

Total execution time 3 milliseconds.

Capture qui indique que mon code est W3C valide sur le fichier testé.

2. Précisez les moyens utilisés :

IDE :Intellij, Maven (gestionnaire de dépendances) : <https://maven.apache.org>, Wamp pour serveur apache (http) + mysql + php : <https://www.wampserver.com/>, **Versioning**: Git : <https://git-scm.com/>, Github : <https://github.com/>, **Portabilité** : Docker : <https://www.docker.com/>, Dockerhub : <https://hub.docker.com/>, **Validateur de qualité du code** : <https://validator.w3.org/>, <https://lighthouse-metrics.com/>

3. Avec qui avez-vous travaillé ?

Configuration effectuée individuellement au début de la formation à LaPlateforme. Les outils ont été sélectionnés de manière identique pour l'ensemble de la promotion afin d'assurer un environnement de travail cohérent entre les apprenants.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Mise en place de l'environnement de développement

Période d'exercice ► Du 16/06/2025 au 18/06/2025

5. Informations complémentaires (facultatif)

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ► Maquetter des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Au cours du projet « MarsAI » (plateforme de courts-métrages générés par IA), j'ai dû, avec mon équipe, concevoir et maquetter une interface utilisateur web et web mobile. Plusieurs étapes préalables sont nécessaires avant la phase de conception.

Dans l'ordre, nous avons d'abord défini les *user stories*, le cahier des charges, le parcours utilisateur, puis la liste des pages et écrans.

Les *user stories* permettent d'identifier les besoins des utilisateurs et de définir les fonctionnalités attendues. Elles suivent une structure générique : « En tant que X, je veux Y afin de Z ».

Par exemple : « En tant que vendeur, je veux accéder à un tableau de bord de mon chiffre d'affaires afin de mieux piloter mon activité ».

Le cahier des charges formalise les contraintes du projet. Il définit les fonctionnalités obligatoires du MVP (Minimum Viable Product), les règles à respecter, ainsi que certains comportements attendus, comme les messages d'erreur ou les restrictions fonctionnelles.

Le parcours utilisateur décrit les interactions et la navigation entre les différents écrans. Par exemple : page d'accueil → connexion → tableau de bord → consultation des résultats.

Ces étapes permettent ensuite de définir la liste des pages et écrans nécessaires au projet. À partir de cette base, nous réalisons le zoning, les wireframes, puis la maquette.

Pour cela, j'ai utilisé Figma au sein d'une équipe de 5 personnes. Bien qu'il existe des alternatives telles que Sketch, Adobe XD ou PenPot, Figma a été retenu pour sa facilité d'accès, ses fonctionnalités collaboratives en temps réel et la richesse de ses ressources partagées.

Dès la phase de conception, je veille à respecter les normes d'accessibilité (WCAG : *Web Content Accessibility Guidelines* et RGAA : *Référentiel général d'amélioration de l'accessibilité*), afin de garantir une interface utilisable par tous, y compris les personnes en situation de handicap (navigation clavier, contrastes suffisants, alternatives textuelles, structure sémantique cohérente).

Je prends également en compte les exigences du RGPD en limitant la collecte de données personnelles, en informant clairement les utilisateurs (cookies, finalité des données) et en prévoyant les pages obligatoires telles que les mentions légales, la politique de confidentialité et une page de contact accessible.

Enfin, j'intègre des principes d'éco-conception afin de réduire l'impact environnemental de l'interface : simplification des écrans, limitation des ressources lourdes, optimisation des images et réduction des requêtes.

Zoning

Le zoning constitue la première étape de la conception. Il consiste à découper la page en grandes zones fonctionnelles (header, contenu principal, sidebar, footer), sans entrer dans le détail graphique. Cette étape permet de définir rapidement la structure et la hiérarchie de l'information.

Wireframe :

Le wireframe, ou maquette fonctionnelle, est un schéma représentant la structure d'une interface utilisateur. Il permet de positionner les zones et composants principaux avant la phase de design graphique.

Dans le cadre du projet :

- Nous avons défini des résolutions cibles : 393 px (mobile, type iPhone 14), 768 px (tablette) et 1440 px (desktop), selon une approche mobile-first.
- Nous avons placé des éléments fonctionnels (boutons, navigation) en nous inspirant de patterns UX existants.
- Nous avons utilisé du contenu temporaire (*Lorem Ipsum*) pour les textes.
- Nous avons utilisé des images génériques pour les illustrations.

Chaque page a été nommée et structurée.

Mockup (maquette haute fidélité):



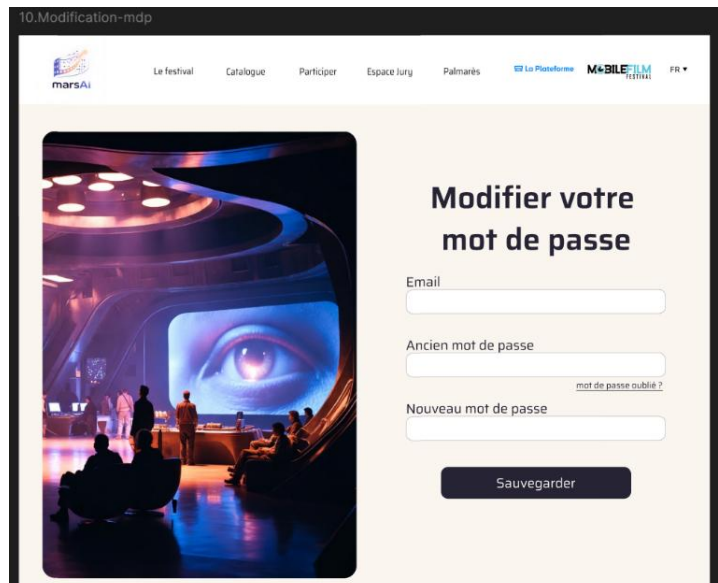
*Exemple de Wireframe
Mobile pour la page
« profil-réalisateur ».*

À partir du wireframe et en respectant la charte graphique du projet (tons sombres, dégradés bleutés, identité MarsAI), nous avons :

- Défini la typographie, les tailles de police et le style des boutons,
- Créé des styles réutilisables (couleurs, textes, espacements),)
- Conçu des composants réutilisables (header, footer, barre de navigation),
- Intégré les éléments finaux (logos, images, contenus, pages réglementaires),
- Assuré la cohérence graphique globale tout en respectant les principes d'accessibilité.

Text styles	
Ag	h1-txt-[mobile] · 32/Auto
Ag	h2-txt-[mobile] · 24/Auto
Ag	h3-txt-[mobile] · 20/Auto
Ag	body-txt-[mobile] · 16/Auto
Ag	small-txt-[mobile] · 14/Auto
Ag	h1-txt-[desktop] · 64/Auto
Ag	h2-txt-[desktop] · 40/Auto
Ag	h3-txt-[desktop] · 28/Auto
Ag	body-txt-[desktop] · 18/Auto
Ag	small-txt-[desktop] · 14/Auto

Exemple : typographie définie et réutilisable



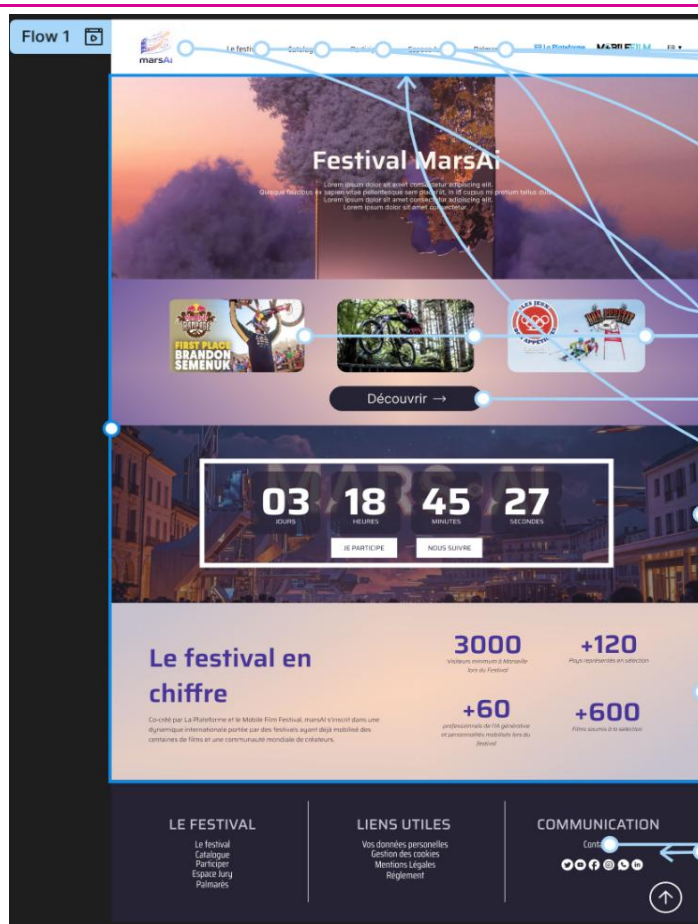
Mockup d'une page de modification de mot de passe

Prototype

Le prototype consiste à :

- relier chaque bouton et élément interactif aux différentes pages selon le parcours utilisateur défini,
- construire un circuit de navigation cohérent, intuitif et accessible.

Maquette pour la home page de MarsAi



2. Précisez les moyens utilisés :

-Word cahier des charges, Figma, Discord pour la communication, Github project pour l'organisation.

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. Les maquettes ont été réalisées collaborativement sur Figma, ce qui a permis à tous les membres de l'équipe de commenter et valider les choix de design avant le développement.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Projet MarsAi – Conception des interfaces

Période d'exercice ► Du 12/01/2026 au 16/01/2026

5. Informations complémentaires (facultatif)

Cliquez ici pour taper du texte.

Activité-type ¹

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 ► Réaliser des interfaces utilisateur statiques web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du développement de la partie front-end, j'ai réalisé la conception et l'intégration d'interfaces utilisateur statiques pour des applications web et web mobile telles que « BankApplication » ou « Media-Tech »

J'utilise un IDE tel que Visual Studio Code et j'organise mes projets en séparant les fichiers **HTML** et **CSS** afin de garantir une meilleure maintenabilité et une meilleure évolutivité du code. Selon la complexité du projet, je peux également utiliser **SCSS** afin de structurer les styles de manière plus modulaire et réutilisable.

Je mets en place l'architecture suivante :

- un layout global servant de base commune aux pages,
- des composants réutilisables (header, footer, navigation).

Concernant la structure HTML, chaque page est composée de balises ouvrantes et fermantes `< >` `</>`, certaines étant auto-fermantes comme `<img>` ou `<link>`. Les deux principales sections sont `<head>` et `<body>`.

Le `<head>` contient les éléments techniques : encodage des caractères (UTF-8), titre de la page, métadonnées SEO, imports CSS et JavaScript (avec l'attribut `defer` lorsque le script est chargé dans le `<head>` afin d'éviter de bloquer le rendu de la page), ainsi que le favicon.

```
<script src="app.js" defer></script>
```

Le `<body>` correspond au contenu de la page et est généralement structuré en trois parties :

- `<header>` : navigation, logo, accès rapides et informations principales,
- `<main>` : contenu principal (textes, images, articles, vidéos, formulaires, tableaux),
- `<footer>` : informations complémentaires (contact, mentions légales, newsletter, liens secondaires).

Dans une logique d'accessibilité et de référencement, j'utilise systématiquement des attributs `alt` pour les images, ainsi que des attributs `ARIA` lorsque nécessaire, par exemple :

`<button aria-label="Fermer">X</button>` afin de décrire les interactions pour les technologies d'assistance.

```

<> dashboard.html > html > body > header.header
1  <!doctype html>
2  <html lang="fr">
3  <head>
4    <meta charset="utf-8">
5    <meta name="viewport" content="width=device-width, initial-scale=1">
6    <title>MarsAI - Dashboard</title>
7
8    <link href="https://fonts.googleapis.com/css2?family=Saira:wght@400;600;700&display=swap" rel="stylesheet">
9    <link rel="stylesheet" href="styles.css">
10 </head>
11
12 <body>
13
14   <header class="header" role="banner">
15
16     <nav class="nav" aria-label="Navigation principale">
17       <a href="#" aria-current="page">Dashboard</a>
18       <a href="#">Projets</a>
19       <a href="#">Analyse</a>
20       <a href="#">Profil</a>
21     </nav>
22   </header>
23
24   <main class="container" role="main">
25
26     <section class="hero" aria-labelledby="title-dashboard">
27       <h1 id="title-dashboard">Tableau de bord</h1>
28       <p>Suivi des générations de courts-métrages IA et statistiques globales.</p>
29     </section>
30
31     <section class="grid" aria-label="Statistiques principales">
32

```

Exemple d'une page HTML, montrant structure + balises aria (accessibilité)

Les pages sont structurées en conteneurs parents et éléments enfants. Le CSS permet ensuite de mettre en forme ces éléments et de gérer leur comportement.

J'utilise au maximum les balises sémantiques : <header>, <nav>, <main>, <section>, <article>, <footer>. Cette utilisation améliore l'accessibilité (lecteurs d'écran) et le référencement naturel (SEO).

J'ai également créé des formulaires complets avec les balises <form>, <input>, <label>, <select>, <textarea> et <button>, en respectant les bonnes pratiques d'accessibilité (liaison for/id, attribut required, types adaptés).

```

13     <section aria-labelledby="login-title">
14         <h1 id="login-title">Connexion</h1>
15
16         <form action="#" method="post" novalidate>
17
18             <div>
19                 <label for="email">Adresse email</label>
20                 <input
21                     type="email"
22                     id="email"
23                     name="email"
24                     required
25                     autocomplete="email"
26                 >
27             </div>
28
29             <div>
30                 <label for="password">Mot de passe</label>
31                 <input
32                     type="password"
33                     id="password"
34                     name="password"
35                     required
36                     autocomplete="current-password"
37                 >
38             </div>
39
40             <button type="submit">Se connecter</button>
41
42         </form>
43     </section>

```

Formulaire avec validations frontend

Concernant la partie CSS, ce langage permet de mettre en forme les éléments HTML : couleurs, tailles, polices, espacements, ainsi que la mise en page via Flexbox et Grid.

Il permet également de gérer le positionnement (margin, padding, gap), les comportements visuels et les animations (transitions, transformations).

J'utilise aussi des variables CSS afin de standardiser les propriétés suivantes :

- couleurs,
- espacements (margin, padding, gap),
- tailles (width, height, font-size),
- border-radius,
- ombres,
- durées d'animation et de transition,

- dégradés et polices.

```
1  :root {
2    /* ===== TYPOGRAPHY ===== */
3    --font-main: "Saira", sans-serif;
4
5    --font-size-sm: 12px;
6    --font-size-md: 14px;
7    --font-size-lg: 16px;
8    --font-size-xl: 22px;
9    --font-size-xxl: 26px;
10
11   --font-weight-regular: 400;
12   --font-weight-medium: 500;
13   --font-weight-semi: 600;
14   --font-weight-bold: 700;
```

Mes variables css

Cette approche améliore :

- la lisibilité et la maintenabilité du code,
- la rapidité de développement,
- la cohérence visuelle globale,
- la conformité à la charte graphique.

Je travaille en mobile-first : je pars des écrans les plus petits vers les plus grands. J'utilise des media queries adaptées aux breakpoints suivants :

- 767 px : mobile,
- 1024 px : tablette / petit laptop,
- 1280 px : desktop standard,
- 1536 px : grands écrans.

```
/* TABLETTE */

@media (min-width: 768px) {

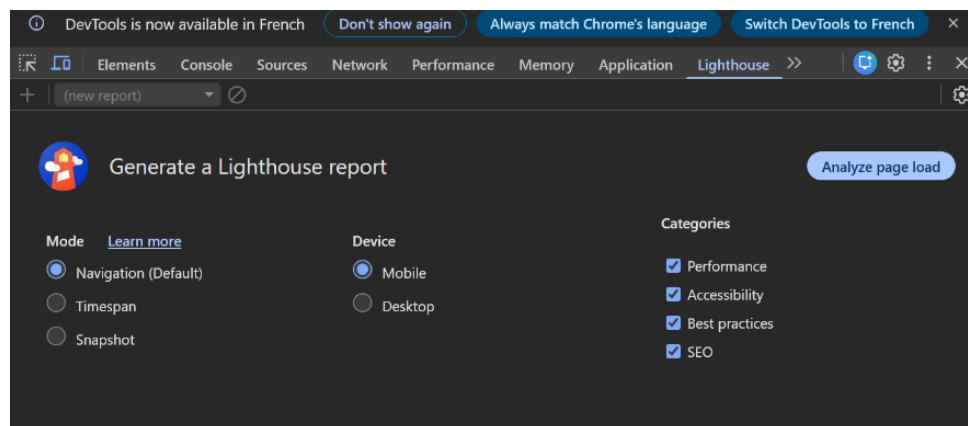
  .header {
    flex-direction: row;
    justify-content: space-between;
    text-align: left;
  }

  .nav {
    flex-direction: row;
    gap: var(--space-md);
  }
}
```

Exemple de @mediaqueries. On est sur le format mobile jusqu'à 768px de largeur (Dans ce cas on passe au format tablette)

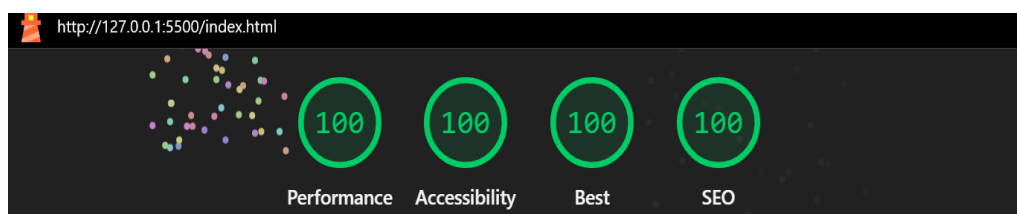
J'utilise ensuite Lighthouse afin d'analyser :

- les performances,
- l'accessibilité,
- le référencement SEO,
- les bonnes pratiques.



J'indique ce que je veux tester par lighthouse

Pour cela j'appuie sur F12 dans mon navigateur chrome pour ouvrir les outils de développement et vais dans l'onglet Lighthouse, je génère ensuite un rapport selon les critères sélectionnés.



Score de 100 validé par lighthouse

2. Précisez les moyens utilisés :

HTML5 (balises sémantiques, formulaires), CSS3 (Flexbox, Grid, transitions, dégradés, variables CSS), media queries pour le responsive design, Visual Studio Code

3. Avec qui avez-vous travaillé ?

Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme, sous la supervision des formateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Exercices de formation – Intégration HTML/CSS

Période d'exercice ► Du 23/06/2025 au 27/06/2025

5. Informations complémentaires (facultatif)

*Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°4 ► Développer la partie dynamique des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ma formation, j'ai mis en place des fonctionnalités dynamiques en JavaScript natif (vanilla JS), sans utiliser de framework. L'objectif était de comprendre les bases du comportement interactif côté navigateur avant d'utiliser des solutions plus avancées.

Gestion des événements

J'ai travaillé avec les événements JavaScript via `addEventListener`, qui permet de détecter les actions de l'utilisateur sur une page web (clic, scroll, saisie clavier, survol, etc.) et d'exécuter une fonction en réponse.

La structure utilisée est la suivante :

`element.addEventListener("événement", fonctionCallback)`

J'ai notamment utilisé ce principe pour rendre une interface plus interactive, par exemple sur un carrousel de projets. J'ai capté l'événement `wheel` afin de permettre un défilement horizontal à l'aide de la molette. L'objet événement (`e`) permet d'accéder à des informations comme la direction du scroll (`e.deltaY`) et de bloquer le comportement natif du navigateur avec `e.preventDefault()`.

```
1  const actionBtn = document.querySelector("#actionBtn");
2
3  actionBtn.addEventListener("click", () => {
4    actionBtn.classList.toggle("active");
5
6    if (actionBtn.classList.contains("active")) {
7      actionBtn.textContent = "Activé";
8    } else {
9      actionBtn.textContent = "Désactivé";
10   }
11 });
```

Exemple d'utilisation de `addEventListener()` pour gérer un événement click en JavaScript.

Menu hamburger (navigation mobile)

J'ai également développé un menu de navigation adapté au mobile sous forme de bouton "burger".

Son fonctionnement repose sur plusieurs étapes :

- sélection des éléments du DOM avec `document.querySelector()`,
- ajout d'un écouteur sur l'événement `click`,
- modification dynamique des classes CSS via `classList.toggle()`.

Lors du clic, une classe est ajoutée ou retirée sur le menu. Cette classe permet, côté CSS, de basculer l'affichage (par exemple de display: none à display: flex) et de déclencher une animation d'ouverture.

```
1  const burgerButton = document.querySelector("#burger");
2  const navMenu = document.querySelector("#menu");
3
4  burgerButton.addEventListener("click", () => {
5    |   navMenu.classList.toggle("active");
6  });
```

Gestion de l'ouverture du menu hamburger avec addEventListener() et classList.toggle()

Manipulation du DOM

Le DOM (Document Object Model) représente la structure de la page HTML dans le navigateur. JavaScript permet de le modifier en temps réel.

J'ai utilisé différentes méthodes pour interagir avec cette structure :

- modification du contenu avec textContent,
- gestion des classes avec classList,
- création et insertion d'éléments avec createElement et appendChild,
- suppression d'éléments avec removeChild,
- modification d'attributs HTML.

Ces manipulations permettent de rendre une interface totalement dynamique sans rechargement de page.

```
1  const loadButton = document.querySelector("#loadData");
2  const output = document.querySelector("#output");
3
4  loadButton.addEventListener("click", async () => {
5    |   output.textContent = "Chargement en cours...";
6
7    |   try {
8    |     |   const response = await fetch("https://jsonplaceholder.typicode.com/todos/1");
9    |
10   |     |   if (!response.ok) {
11   |     |   |   throw new Error("Erreur réseau");
12   |     |   }
13   |
14   |     |   const data = await response.json();
15   |
16   |     |   output.textContent = `Tâche : ${data.title}`;
17   |   } catch (error) {
18   |     |   output.textContent = "Impossible de charger les données";
19   |   }
20  });
```

Récupération de données depuis une API avec fetch() et mise à jour dynamique du DOM.

Outils et technologies utilisés ensuite

Après avoir consolidé les bases en JavaScript natif, j'ai travaillé avec des technologies plus avancées dans d'autres projets.

J'ai utilisé React pour structurer les interfaces en composants réutilisables et mieux organiser la gestion de l'état de l'application.

2. Précisez les moyens utilisés :

JavaScript ES6+ (vanilla) addEventListener, manipulation du DOM (querySelector, classList, createElement)

React.js. Visual Studio Code

3. Avec qui avez-vous travaillé ?

Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme, sous la supervision des formateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ▶ LaPlateforme (centre de formation)

Chantier, atelier, service ▶ Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme

Période d'exercice ▶ Du 07/07/2025 au 18/07/2025

5. Informations complémentaires (facultatif)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet "**MarsAi** » j'ai conçu et implémenté une base de données relationnelle afin de structurer les données de l'application et garantir leur intégrité.

Modélisation de la base de données (MERISE) (Voir annexe).

J'ai commencé par la conception du modèle de données en suivant la méthode MERISE, qui permet de structurer la base en plusieurs niveaux : **MCD (Modèle Conceptuel de Données)** : définition des entités et relations métier, **MLD (Modèle Logique de Données)** : transformation en tables relationnelles, **MPD (Modèle Physique de Données)** : adaptation au SGBD MySQL

Cette étape m'a permis de définir la structure globale de la base avant implémentation.

Mise en place du SGBD

J'ai utilisé **MySQL** comme système de gestion de base de données relationnelle, installé localement avec son port par défaut (3306).

Pour administrer la base, j'ai utilisé **MySQL Workbench**, qui permet de :

- visualiser la structure des tables
- exécuter des requêtes SQL
- importer et tester le schéma de données

Création des tables et structure SQL

Le modèle physique de données (MPD) a été directement implémenté sous forme de scripts SQL MySQL (CREATE TABLE), constituant ainsi la structure finale de la base de données.

Pour chaque table, j'ai défini :

- les types de données adaptés (INT, VARCHAR, TEXT, DATETIME, etc.)
- les contraintes d'intégrité (NOT NULL, UNIQUE, DEFAULT, ENUM)
- les clés primaires (PRIMARY KEY) avec auto-incrémentation

Relations et intégrité des données

J'ai mis en place les relations entre les différentes tables à l'aide de clés étrangères (FOREIGN KEY), garantissant l'intégrité référentielle de la base de données.

Cela permet d'assurer la cohérence des données entre les entités (par exemple entre les films, les catégories et les utilisateurs) et d'éviter les incohérences ou données orphelines.

Des index ont également été ajoutés sur certaines colonnes afin d'optimiser les performances des requêtes.

Gestion des données et sécurité

Des scripts d'insertion (INSERT INTO) ont été utilisés afin de tester la base de données avec des données réalistes et de valider son bon fonctionnement.

Dans une logique de sécurité, les mots de passe sont stockés sous forme hashée et jamais en clair dans la base de données.

Gestion des droits utilisateurs

Un utilisateur MySQL dédié au projet a été créé avec des droits restreints (SELECT, INSERT, UPDATE, DELETE) uniquement sur la base de données concernée.

Cette approche permet de limiter les risques en cas de compromission de l'application et évite l'utilisation du compte administrateur (root).

```
1 • CREATE USER 'marsaiAdmin'@'localhost' IDENTIFIED BY 'strongpassword';
2
3 • GRANT SELECT, INSERT, UPDATE, DELETE
4   ON marsai.*
5   TO 'marsaiAdmin'@'localhost';
6
7 • FLUSH PRIVILEGES;
```

Création d'un user admin + attribution de ses droits pour gérer la base de données « MarsAi ».

2. Précisez les moyens utilisés :

SGBD : MySQL, Langage utilisé : SQL (création de tables, requêtes, insertions, gestion des droits), Outil d'administration : MySQL Workbench. Méthode de conception : MERISE (MCD, MLD, MPD)

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. La conception de la base de données a été réalisée collectivement lors de sessions de travail en équipe.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation).

Chantier, atelier, service ► Projet MarsAi – Module base de données

Période d'exercice ► Du 19/01/2026 au 23/01/2026

5. Informations complémentaires (facultatif)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet « TeacherApp », j'ai développé les composants permettant au back-end d'accéder à une base de données relationnelle. L'application a été développée avec Java et Spring Boot. Maven a été utilisé pour gérer les dépendances et assurer la reproductibilité de la configuration du projet.

Les principales dépendances utilisées sont Spring Data JPA, Spring Validation, MySQL Connector et Lombok. Lombok est une bibliothèque qui permet de réduire le code en générant automatiquement certains éléments comme les getters, setters, constructeurs ou encore toString. Par exemple, l'annotation @Data regroupe @Getter, @Setter, @ToString et d'autres fonctionnalités, ce qui simplifie la lecture et la maintenance du code.

- La validation des données est assurée grâce à Jakarta Validation (Bean Validation). Elle permet de définir des contraintes directement au niveau de l'entité, comme @NotBlank, @Email ou @Size. Ces annotations garantissent l'intégrité des données avant leur persistance en base de données et permettent de renvoyer des erreurs claires en cas de données invalides.

Connexion à la base de données (MySQL Connector)

La connexion à la base de données MySQL est configurée dans le fichier src/main/resources/application.properties. J'y renseigne notamment l'URL de connexion JDBC, le nom d'utilisateur, le mot de passe, le pilote MySQL ainsi que les paramètres de JPA et Hibernate.

Spring Boot utilise le pilote MySQL Connector pour établir la communication avec la base de données. JDBC constitue l'API Java permettant cet accès. Hibernate traduit ensuite les opérations réalisées sur les objets Java en requêtes SQL exécutées par MySQL.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

Dépendance mysql-connector-j placé dans le pom.xml

Je configure la connexion dans src/main/resources/application.properties


```

1  spring:
2  datasource:
3  url: jdbc:mysql://localhost:3306/teacherapp
4  username: ${DB_USER}
5  password: ${DB_PASSWORD}

```

Configuration de la connexion à la base de données mysql (3306)

La chaîne d'accès aux données est la suivante :

Application → Spring Data JPA → JPA → Hibernate → JDBC → MySQL

Modélisation des données : couche Entity

À partir du modèle de données, j'ai créé des classes Java représentant les tables de la base de données. Ces classes sont placées dans une couche model ou entity et sont annotées avec @Entity

```

12  // Lombok annotation that automatically generates getters, setters
13  // + toString(), equals(), and hashCode() methods.
14  @Data  @ kael +1
15
16  @NoArgsConstructor // Generates a no-argument constructor required by JPA (Hibernate) for entity instantiation.
17
18  @Entity // Marks this class as a JPA entity mapped to a database table.
19
20  @Table(name = "teacher") // Specifies the name of the database table associated with this entity.
21  public class Teacher {

```

Annotations de ma classe Entité « Teacher »

```

23  @Id // Primary key
24  @GeneratedValue(strategy = GenerationType.IDENTITY) // Automatically generates the ID using the database identity column (auto-increment).
25  @ private Integer id;
26
27  @NotBlank // Must be not blank / not null. Not Blank is preferred for Strings attributes.
28  @Size(min = 2, max = 50) // Ensures valid length constraints.
29  @ private String firstName;
30
31  @NotBlank
32  @Size(min = 2, max = 50)
33  @ private String lastName;
34
35  @Column(unique = true, nullable = false)
36  @Email // Validates that the value follows a valid email format.
37  @ private String email;
38
39  @ManyToOne // Defines a many-to-one relationship (many teachers belong to one school).
40  @JoinColumn(name = "school_id") // Specifies the foreign key column in the teacher table.
41  private School school;
42  }

```

Annotations côté backend/SpringBoot permettant à JPA de créer les tables (@Id = Clé primaire)

Chaque entité contient les attributs correspondant aux colonnes de la table, une clé primaire identifiée avec @Id, ainsi que les relations éventuelles entre les entités. Les annotations JPA permettent de réaliser le mapping objet-relationnel entre les classes Java et les tables SQL.

J'ai également ajouté des règles de validation sur certaines données saisies par l'utilisateur grâce à Jakarta Validation. Par exemple, les annotations @NotBlank, @Email et @Size permettent de contrôler la validité des données avant leur enregistrement en base de données. Cela évite la persistance de données incomplètes ou incorrectes et permet de retourner des messages d'erreur adaptés au client.

Couche Repository

J'ai créé une couche repository dédiée à l'accès aux données. Chaque repository est une interface étendant JpaRepository.

```
 public interface TeacherRepository extends JpaRepository<Teacher, Integer> {
```

Cette interface fournit automatiquement les principales opérations CRUD(save, findById, findAll, delete), sans avoir à écrire les requêtes SQL correspondantes : enregistrement avec save, recherche par identifiant avec findById, récupération d'une liste avec findAll et suppression avec delete.

Cette organisation sépare l'accès aux données de la logique métier présente dans la couche service. Le code est ainsi plus lisible, plus facile à maintenir et à tester.

Certaines méthodes retournent un Optional<T>. Cet objet permet de représenter explicitement l'absence éventuelle d'un résultat, sans manipuler directement la valeur null. Cela limite les risques de NullPointerException et oblige le développeur à traiter le cas où l'entité recherchée n'existe pas.

Méthodes de requêtes personnalisées

J'ai utilisé les Query Methods proposées par Spring Data JPA afin de réaliser des recherches simples à partir du nom des méthodes déclarées dans les repositories.

Par exemple :

- findByEmail(String email)
- existsByEmail(String email)
- findByLastName(String lastName)

Spring Data JPA interprète le nom de ces méthodes et génère automatiquement la requête nécessaire. Cette solution permet de créer rapidement des recherches courantes tout en conservant un code lisible.

Pour des besoins plus spécifiques, il est également possible d'utiliser JPQL avec l'annotation @Query. Contrairement au SQL natif, JPQL s'appuie sur les noms des entités Java et de leurs attributs, plutôt que sur les noms des tables et colonnes de la base de données.

```
//Query Methods:
Optional<Teacher> findByLastName(String lastName);
boolean existsByEmail(String email); 1 usage 2 kael
```

Exemple de Query Methods

JPQL (Java Persistence Query Language)

Lorsque les requêtes deviennent plus complexes, il est possible d'utiliser JPQL via l'annotation @Query.

JPQL permet d'écrire des requêtes en se basant sur les entités Java et non sur les tables SQL.

```
// JPQL query:
// Searches for all Teacher entities whose lastName matches the provided value.
@Query("SELECT t FROM Teacher t WHERE t.lastName = :lastName") 1 usage 2 kael
List<Teacher> findTeachersByLastName(String lastName);
```

Exemple pour retourner une liste des enseignants ayant pour lastName « lastName » (à remplacer par la valeur souhaitée).

Native Query (SQL natif)

Dans certains cas, il peut être nécessaire d'écrire du SQL pur via nativeQuery = true.

Exemple :

```
// Native SQL query:
// Allows the use of database-specific features that are not available in JPQL.
@Query( 1 usage 2 kael
    value = "SELECT * FROM teacher WHERE email = :email",
    nativeQuery = true
)
Teacher findByEmailNative(String email);
```

Exemple de requête SQL native avec nativeQuery = true

Retourne un objet « teacher » ayant pour email : « email » (à remplacer avec la valeur souhaitée).

Accès aux données NoSQL avec MongoDB

En complément de la base relationnelle MySQL j'ai étudié et mis en pratique l'accès aux données NoSQL avec MongoDB.

MongoDB est une base de données orientée documents. Contrairement à MySQL, les données ne sont pas organisées dans des tables liées par des clés étrangères, mais dans des collections contenant des documents au format BSON. Le BSON est un format binaire proche du JSON, permettant de stocker des données structurées, imbriquées ou variables.

Par exemple, un document MongoDB peut représenter un utilisateur, un produit ou un article avec ses informations directement regroupées dans un même objet. Il est également possible d'intégrer des tableaux ou des sous-documents sans devoir créer plusieurs tables relationnelles.

MongoDB est particulièrement adapté lorsque la structure des données peut évoluer fréquemment, lorsque les données sont semi-structurées ou lorsqu'une entité contient naturellement des informations imbriquées. En revanche, une base relationnelle comme MySQL reste plus adaptée lorsque les données comportent de nombreuses relations complexes et nécessitent une forte intégrité référentielle.

Avec Spring Boot, l'accès à MongoDB peut être réalisé grâce à la dépendance Spring Data MongoDB. La connexion est configurée dans le fichier `application.properties` avec une URL de connexion MongoDB.

Les documents sont représentés dans l'application par des classes Java annotées avec `@Document`. Cette annotation indique que la classe correspond à une collection MongoDB. Chaque document possède généralement un identifiant annoté avec `@Id`.

```
@Document(collection = "consultations")
```

J'ai ensuite créé une interface repository étendant `MongoRepository`. Cette interface fournit des méthodes CRUD similaires à celles de Spring Data JPA, telles que `save`, `findById`, `findAll` et `deleteById`.

```
public interface ConsultationRepository extends MongoRepository<Consultation, String>
```

Il est également possible de créer des méthodes de recherche à partir de leur nom, par exemple `findByEmail(String email)` ou `findByCategory(String category)`. Spring Data MongoDB génère alors la requête adaptée à la collection concernée.

Cette mise en pratique m'a permis de comparer les deux approches. MySQL utilise un schéma structuré, des tables, des relations et des contraintes. MongoDB privilégie une structure plus souple, fondée sur des documents JSON pouvant contenir des données imbriquées. Le choix dépend donc des besoins fonctionnels, de la structure des données et des contraintes du projet.

2. Précisez les moyens utilisés :

IntelliJ IDEA / Java 17 / Spring Boot, MySQL + MongoDB, Spring Data JPA + MongoRepository, MySQLWorkbench + MongoDB Compass, Maven, Docker

3. Avec qui avez-vous travaillé ?

Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme, sous la supervision des formateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► HealthCare – couche Repository

Période d'exercice ► Du 13/05/2026 au 23/05/2026

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°7 ► Développer des composants métier coté serveur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Sur le projet **MarsAI**, j'ai développé les composants métier côté serveur au sein de la couche service. Un composant métier correspond à un ensemble de règles et de traitements liés à une fonctionnalité précise de l'application, indépendamment de la couche HTTP (controller) et de la couche d'accès aux données (repository).

Composant métier – inscription d'un utilisateur (register) : Le service d'inscription représente un composant métier complet car il enchaîne plusieurs règles de gestion :

```
export const register = async ({ email, password, name }) => {
  const exists = await userRepository.emailExists(email);
  if (exists) throw Object.assign(new Error("Un utilisateur avec cet email existe déjà"), { status: 400 });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await userRepository.create({ email, password: hashedPassword, name });

  await userRepository.assignRole(user.id, 1);

  const rolesRaw = await userRepository.getRoleIds(user.id);
  const roles = normalizeRoleIds(rolesRaw);

  const token = jwt.sign({ userId: user.id, email: user.email, roles }, SECRET, { expiresIn: JWT_EXPIRES_IN });

  const { password: _pw, ...userWithoutPassword } = user;
  return { user: userWithoutPassword, token };
};
```

Implémentation du processus d'inscription d'un utilisateur avec hachage BCrypt et génération d'un jeton JWT

1. Contrôle de l'unicité de l'email

Appel à `userRepository.emailExists(email)` afin de vérifier qu'aucun compte n'existe déjà avec cette adresse email. Si un utilisateur est trouvé, une erreur avec un code HTTP 400 est retournée.

2. Hashage du mot de passe

Avant toute sauvegarde en base, le mot de passe est chiffré à l'aide de `bcrypt`. Le mot de passe en clair n'est jamais stocké.

3. Création de l'utilisateur

Appel à `userRepository.create()` avec les données validées et le mot de passe hashé.

4. Attribution du rôle par défaut

Le rôle utilisateur est automatiquement assigné via `userRepository.assignRole(user.id, 1)`.

5. Génération du token JWT

Un token JWT est généré à partir des informations de l'utilisateur (id, email, rôles), signé avec une clé secrète stockée dans les variables d'environnement.

6. Retour sécurisé des données

Le mot de passe est retiré de l'objet renvoyé avant transmission au controller afin d'éviter toute exposition de données sensibles.

Rôle du JWT dans ce composant :

Le JWT (JSON Web Token) sert uniquement à permettre l'authentification immédiate après l'inscription, sans nouvelle connexion.

La véritable logique métier reste le processus d'inscription et ses règles de validation, et non le token lui-même.

2. Précisez les moyens utilisés :

Node.js, Express.js, architecture en couches (service → repository), bcrypt pour le hashage des mots de passes, jsonwebtoken (JWT) pour la génération du token d'authentification, variables d'environnement (JWT_SECRET, JWT_EXPIRES_IN)

3. Avec qui avez-vous travaillé ?

En équipe de 5 développeurs à LaPlateforme. J'étais le référent technique sur la partie back-end et j'ai développé l'ensemble des services métier.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► Projet MarsAI – Composants métier back-end

Période d'exercice ► Du 16/02/2026 au 20/02/2026

5. Informations complémentaires (facultatif)

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°8 ► Documenter le déploiement d'une application dynamique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour documenter le déploiement d'une application dynamique web ou web mobile comme « Healthcare » je passe par le Readme.md du dépôt GitHub. L'objectif est de permettre à un tiers de déployer l'application sans dépendance à mon environnement local.

L'application repose sur une architecture composée de plusieurs couches :

- un front-end (HTML/CSS/JavaScript ou React selon modules),
- un back-end en Java Spring Boot,
- une base de données MySQL,
- un module MongoDB pour certaines données non relationnelles.

Variables d'environnement

J'ai utilisé des variables d'environnement pour isoler les informations sensibles (identifiants, URLs, mots de passe).

Ces variables sont stockées dans un fichier .env, non versionné grâce à .gitignore.

Un fichier .env.example est fourni afin de permettre à un autre développeur de recréer l'environnement.

```
1 # Variables d'environnement (données sensibles)
2 .env
3
4 # Fichier système macOS
5 .DS_Store
6
7 # Dossier de build (code compilé)
8 dist/
9
10 # Dossier de production généré
11 build/
12
13 # Logs
14 *.log
15
16 # IDE JetBrains (IntelliJ IDEA, etc.)
17 .idea/
18 *.iml
19 out/
```

Exemple d'un fichier .gitignore

```
1 # DATABASE CONFIGURATION
2 DB_HOST=localhost
3 DB_PORT=3306
4 DB_NAME=my_database
5 DB_USER=root
6 DB_PASSWORD=strong-password
7
8 # BACKEND CONFIGURATION
9 SERVER_PORT=8080
10
11 #Git related configuration
12 Git-API_KEY=your-git-api-key
```

Exemple d'un fichier .env contenant valeurs génériques

Docker et conteneurisation

J'ai conteneurisé les services de l'application via Docker afin de garantir la portabilité du projet.

Chaque micro-service est construit à partir d'un Dockerfile générant un fichier .jar pour le back-end Java.

Le cycle de déploiement est le suivant :

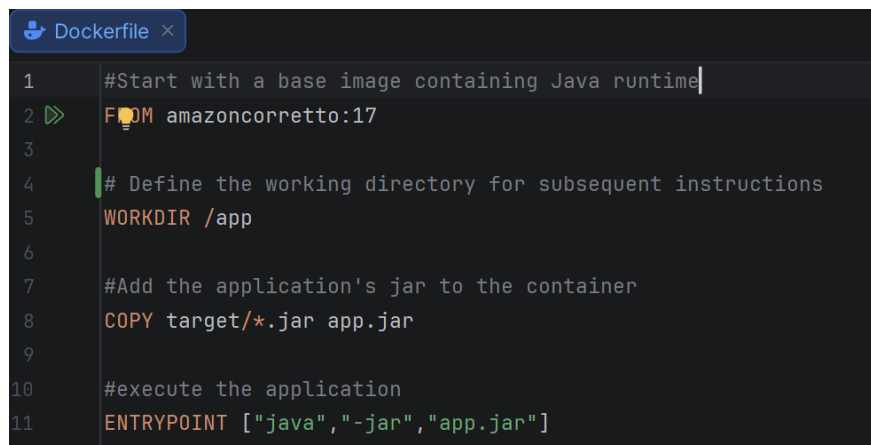
- compilation Maven du projet
- génération du .jar
- création de l'image Docker
- exécution du conteneur

Docker permet d'exécuter l'application dans un environnement isolé partageant le noyau du système hôte, ce qui garantit un fonctionnement identique sur toutes les machines.

Le Dockerfile définit les instructions permettant de construire une image Docker pour le service backend.

Il précise notamment :

- l'image de base (Java JDK),
- la copie du fichier .jar généré par Maven,
- et la commande de lancement de l'application.



```
1 #Start with a base image containing Java runtime
2 FROM amazoncorretto:17
3
4 # Define the working directory for subsequent instructions
5 WORKDIR /app
6
7 #Add the application's jar to the container
8 COPY target/*.jar app.jar
9
10 #execute the application
11 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Exemple d'un dockerfile

Maven est un outil de gestion de projet Java. Il permet d'automatiser le cycle de vie de l'application, notamment :

- la compilation du code source,
- l'exécution des tests,
- la gestion des dépendances,
- la génération du fichier exécutable .jar.

La commande

```
mvn clean package
```


- supprime les anciens fichiers compilés,
- reconstruit entièrement le projet,
- produit un artefact .jar prêt pour le déploiement.

Ce fichier .jar est ensuite utilisé pour la création de l'image Docker du microservice. Je construis ensuite l'image Docker à partir du fichier .jar généré par Maven avec la commande :

Le paramètre -t permet de nommer et taguer l'image, et le . indique que le contexte de build correspond au dossier contenant le Dockerfile.

L'image peut ensuite être exécutée sous forme de conteneur avec Docker. Les conteneurs s'exécutent sur le kernel du système hôte. Sur Windows ou MacOS Docker utilise une couche de virtualisation (WSL2 ou VM Linux) afin de fournir un environnement Linux compatible. Je démarre le conteneur avec cette commande :

```
docker build -t kael/patients .
```

```
docker run patients
```

Orchestration avec Docker Compose

J'ai utilisé Docker Compose pour lancer l'ensemble des services simultanément via un fichier **docker-compose.yml**.

Cela permet :

- de démarrer tous les services en une seule commande,
- de gérer les dépendances entre services,
- de simplifier le déploiement global de l'application.

Exemple d'un docker-compose.yml :

```

86 patients:
87   image: patients
88   container_name: patients
89   mem_limit: 700m
90   restart: on-failure
91   ports:
92   | - "8080:8080"
93   depends_on:
94   | config-server:
95   |   condition: service_healthy
96   | mysql:
97   |   condition: service_healthy
98   | consul:
99   |   condition: service_healthy
100  environment: #Defines environment variables passed to the container
101    SPRING_APPLICATION_NAME: "patients"
102    SPRING_DATASOURCE_URL: "jdbc:mysql://mysql:3306/healthcare"
103    SPRING_CLOUD_CONSUL_HOST: "consul"
104    SPRING_CLOUD_CONSUL_PORT: "8500"
105  networks:
106  | - healthcare-network

```

Exemple de docker-compose.yml

Documentation

Enfin, j'ai rédigé un fichier README.md détaillant : prérequis (Docker, Java, Maven), étapes d'installation, commandes de lancement et structure du projet.

2. Précisez les moyens utilisés :

Git / GitHub, Maven, Docker + Docker Compose, JDK 17, Spring Boot, MySQL, MongoDB, fichiers .env, README.md

3. Avec qui avez-vous travaillé ?

Exercices réalisés individuellement dans le cadre de la formation à LaPlateforme, sous la supervision des formateurs.

4. Contexte

Nom de l'entreprise, organisme ou association ► LaPlateforme (centre de formation)

Chantier, atelier, service ► HealthCare – déploiement

Période d'exercice ► Du 27/05/2026 au 05/06/2026

5. Informations complémentaires (facultatif)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e) [prénom et nom] Théo VITI..... ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur(e) des réalisations jointes.

Fait à : Martigues..... le 02/07/2026.....

pour faire valoir ce que de droit.

Signature :

A handwritten signature in black ink, appearing to be 'Théo VITI', written over a horizontal line.

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

ANNEXES

(Si le RC le prévoit)

Modélisation des données – Méthode Merise (Activité n°5)

Projet MarsAI Festival — MCD / MLD / MPD

1. Modèle Conceptuel de Données (MCD)

Le MCD décrit les entités du domaine métier ainsi que les associations qui les relient, avec leurs cardinalités.

1.1 Entités et attributs

Entité	Attributs
UTILISATEUR	id, nom, email, mdp, bio, fonction, doit_reset_mdp
ROLE	id, nom_role
FILM	id, titre, pays, statut, url_youtube, certification_ia, classification
CATEGORIE	id, nom, description
RECOMPENSE	id, rang, type_prix, nom_prix, annee, montant
LISTE_JURY	id, nom, description
INVITATION	id, email, token, expire_le, accepte_le
PAGE_SITE	id, slug, contenu_fr, contenu_en
PARTENAIRE	id, nom, logo, url, ordre_affichage
EVENEMENT	id, type, titre, date_evenement, lieu
ABONNE_NEWSLETTER	id, email, langue, confirme
TOKEN_RESET_MDP	id, token, expire_le, utilise
LOG_EMAIL	id, email_destinataire, type_email, envoye_le

1.2 Associations et cardinalités

Association	Entité A	Card. A	Entité B	Card. B	Attributs portés
POSSEDER	UTILISATEUR	0,n	ROLE	0,n	—
CLASSER	FILM	0,n	CATEGORIE	0,n	date_assignment
NOTER	UTILISATEUR (jury)	0,n	FILM	0,n	note, commentaire
EVALUER	UTILISATEUR (jury)	0,n	FILM	0,n	statut, motif_refus
VALIDER	UTILISATEUR (admin)	0,1	FILM	0,n	—
REMPORTER	FILM	1,n	RECOMPENSE	1,1	—
CONCERNER	CATEGORIE	0,1	RECOMPENSE	0,n	—
CONTENIR	LISTE_JURY	0,n	FILM	1,n	—
DISTRIBUER	LISTE_JURY	0,n	UTILISATEUR	0,n	assigne_par
CREER	UTILISATEUR	1,1	LISTE_JURY	0,n	—
ENVOYER	UTILISATEUR	1,1	INVITATION	0,n	—
DEMANDER	UTILISATEUR	1,1	TOKEN_RESET_MDP	0,n	—
MODIFIER	UTILISATEUR	0,1	PAGE_SITE	0,n	—
GENERER	FILM	1,1	LOG_EMAIL	0,n	—

2. Modèle Logique de Données (MLD)

Le MLD traduit le MCD en un schéma relationnel : chaque entité et chaque association porteuse de données ou de type n,n devient une table. Les clés étrangères sont notées FK, les clés primaires PK, les clés uniques UK.

Table	Colonnes
ROLES	id (PK), role_name
USERS	id (PK), name, email (UK), password, must_reset_password, profile_picture, role_title, bio, created_at
USER_ROLES	user_id (PK, FK -> USERS), role_id (PK, FK -> ROLES)
PASSWORD_RESET_TOKENS	id (PK), user_id (FK -> USERS), token (UK), expires_at, used
NEWSLETTER_SUBSCRIBERS	id (PK), email (UK), lang, confirmed, confirm_token
CATEGORIES	id (PK), name (UK), description
FILMS	id (PK), title, country, description, status, status_changed_by (FK -> USERS), director_firstname, director_lastname, director_email
FILM_CATEGORIES	film_id (PK, FK -> FILMS), category_id (PK, FK -> CATEGORIES), assigned_by (FK -> USERS), assigned_at

EMAIL_LOGS	id (PK), film_id (FK -> FILMS), recipient_email, email_type, sent_at
AWARDS	id (PK), film_id (FK -> FILMS), category_id (FK -> CATEGORIES), rank, award_type, award_name, year
JURY_LISTS	id (PK), name, description, created_by (FK -> USERS)
JURY_LIST_FILMS	list_id (PK, FK -> JURY_LISTS), film_id (PK, FK -> FILMS)
JURY_LIST_ASSIGNMENTS	id (PK), list_id (FK -> JURY_LISTS), jury_id (FK -> USERS), assigned_by (FK -> USERS)
JURY_ASSIGNMENTS	id (PK), jury_id (FK -> USERS), film_id (FK -> FILMS), list_id (FK -> JURY_LISTS), assigned_by (FK -> USERS), status, refusal_reason
JURY_RATINGS	id (PK), film_id (FK -> FILMS), user_id (FK -> USERS), rating, comment
INVITATIONS	id (PK), email, role_id (FK -> ROLES), invited_by (FK -> USERS), token, expires_at
SITE_PAGES	id (PK), slug (UK), content_fr, content_en, updated_by (FK -> USERS)
PARTNERS	id (PK), name, logo, url, display_order
EVENTS	id (PK), event_type, title, event_date, location
FESTIVAL_CONFIG	id (PK), submission_start, submission_end, event_date

3. Modèle Logique de Données (MPD)

Le MPD correspond à l'implémentation concrète de la base de données dans le SGBD MySQL. Il traduit le modèle logique en structures physiques exploitables par le système.

```

143 DROP TABLE IF EXISTS `films`;
144 CREATE TABLE `films` (
145   `id` int NOT NULL AUTO_INCREMENT,
146
147   -- Film Information
148   `title` varchar(255) NOT NULL,
149   `title_english` varchar(255) DEFAULT NULL COMMENT 'English title for international audiences',
150   `country` varchar(100) DEFAULT NULL COMMENT 'Country of origin',
151   `description` text COMMENT 'Original description in the director''s language',
152   `description_english` text COMMENT 'English description for international audiences',
153   `film_url` varchar(500) DEFAULT NULL COMMENT 'URL to uploaded film file',
154   `youtube_url` varchar(500) DEFAULT NULL COMMENT 'YouTube video URL for public viewing',
155   `poster_url` varchar(500) DEFAULT NULL COMMENT 'Main poster image',
156   `thumbnail_url` varchar(500) DEFAULT NULL COMMENT 'Small thumbnail for lists',
157   `ai_tools_used` text COMMENT 'AI tools used (free text)',
158   `ai_certification` tinyint(1) DEFAULT 0 COMMENT 'Certifies film was made with AI tools',
159   `classification` varchar(50) DEFAULT NULL COMMENT 'Classification du film, ex: G, PG, PG-13, R',
160 )

```